

PATRÓN DE DISEÑO: MEDIADOR (MEDIATOR)

Capítulo 31 — Patrones de Diseño en Java

Traducción técnica al español

© 2004 CRC Press LLC — Traducción al español

1. Descripción

Este patrón fue descrito previamente en GoF95.

En general, las aplicaciones orientadas a objetos consisten en un conjunto de objetos que interactúan entre sí con el propósito de proveer un servicio. Esta interacción puede ser directa (punto a punto) siempre que el número de objetos que se referencian directamente sea muy bajo.

A medida que aumenta el número de objetos, este tipo de interacción directa crea un complejo laberinto de referencias entre objetos, lo que dificulta el mantenimiento de la aplicación. Además, tener un objeto que hace referencia directa a muchos otros objetos reduce considerablemente las posibilidades de reutilización de esos objetos debido al alto acoplamiento.

En tales casos, el Patrón Mediador puede utilizarse para diseñar un modelo de comunicación coordinado para un grupo de objetos, eliminando la necesidad de que los objetos se referencien directamente entre sí.

El Patrón Mediador sugiere abstraer todos los detalles de interacción entre objetos en una clase separada, denominada Mediador, con conocimiento sobre el grupo de objetos. Cada objeto del grupo sigue siendo responsable de ofrecer el servicio para el que fue diseñado, pero los objetos no interactúan directamente entre sí para ese propósito. La interacción entre dos objetos diferentes se enruta a través de la clase Mediador. Todos los objetos envían sus mensajes al mediador, quien a su vez envía mensajes a los objetos apropiados según los requerimientos de la aplicación.

El diseño resultante tiene las siguientes ventajas principales:

- Al mover todo el comportamiento de interacción entre objetos a un objeto separado, resulta más fácil alterar el comportamiento mediante la sustitución del mediador por una de sus subclases con funcionalidad modificada.
- Mover las dependencias entre objetos fuera de los objetos individuales mejora la reutilizabilidad de los objetos.
- Dado que los objetos no necesitan referenciarse directamente entre sí, pueden probarse de manera unitaria con mayor facilidad.

- El bajo grado de acoplamiento resultante permite que las clases individuales sean modificadas sin afectar a otras clases.

2. Mediador versus Fachada

En algunos aspectos, el Patrón Mediador parece similar al Patrón Fachada. La Tabla 1 lista las similitudes y diferencias entre ambos patrones.

Tabla 1 — Mediador versus Fachada

Aspecto	Mediador	Fachada
Propósito	Se usa para abstraer la funcionalidad necesaria de un grupo de objetos con el objetivo de simplificar la interacción entre ellos.	Se usa para abstraer la funcionalidad de un subsistema o sus componentes, con el objetivo de proveer una interfaz simplificada de nivel superior.
Conocimiento mutuo	Todos los objetos interactúan entre sí a través del Mediador. El grupo de objetos conoce la existencia del Mediador.	Los clientes usan la Fachada para interactuar con los componentes del subsistema. La existencia de la Fachada no es conocida por los componentes del subsistema.
Dirección de la comunicación	Dado que el Mediador y todos los objetos registrados en él pueden comunicarse entre sí, la comunicación es bidireccional.	Los clientes pueden enviar mensajes (a través de la Fachada) al subsistema, pero no viceversa, haciendo que la comunicación sea unidireccional.
Posición en la arquitectura	El Mediador puede concebirse como estando en el centro de un grupo de objetos que interactúan.	La Fachada se sitúa entre el cliente y el subsistema.
Impacto del cambio	Usar un Mediador permite cambiar la implementación de cualquiera de los objetos que interactúan sin impacto sobre los otros objetos que solo interactúan a través del Mediador.	Usar una Fachada permite cambiar la implementación del subsistema sin impacto sobre sus clientes, siempre que estos no tengan acceso directo a las clases del subsistema.
Extensibilidad	Al subclasificar el Mediador, el comportamiento de las interrelaciones entre objetos puede extenderse.	Al subclasificar la Fachada, la implementación de la interfaz de nivel superior puede cambiarse.

3. Ejemplo I — Cliente FTP

3.1 Descripción del Escenario

La aplicación de simulación del cliente FTP construida en el capítulo anterior (Patrón Comando) incluye la siguiente lista de controles de interfaz de usuario:

Tabla 2 — Objetos de Interfaz de Usuario y su Funcionalidad

Control UI	Funcionalidad
JList (local)	Muestra el sistema de archivos local.
JList (remoto)	Muestra el sistema de archivos remoto.
JButton Upload	Provee la funcionalidad de subida. Al hacer clic, el archivo seleccionado en el sistema de archivos local se sube al servidor remoto y su nombre se agrega al control JList del sistema de archivos remoto.
JButton Download	Provee la funcionalidad de descarga. Al hacer clic, el archivo seleccionado en el sistema de archivos remoto se descarga al sistema local y su nombre se agrega al control JList del sistema de archivos local.
JButton Delete	Provee la funcionalidad de eliminación. Al hacer clic, el archivo seleccionado en el sistema de archivos local o remoto es eliminado. El JList correspondiente se actualiza en consecuencia.

3.2 Mejoras al Diseño

Consideremos las siguientes mejoras menores a la aplicación existente para hacerla más amigable con el usuario:

- Cuando el UI se muestra por primera vez, todos los botones excepto el botón Exit deben estar deshabilitados.
- Cuando se selecciona un nombre de archivo del control JList que muestra el sistema de archivos local:
 - Los botones Upload y Delete deben habilitarse.
 - Cualquier ítem seleccionado en el sistema de archivos remoto debe deseleccionarse.
 - El botón Download debe deshabilitarse.
- Cuando se selecciona un nombre de archivo del control JList que muestra el sistema de archivos remoto:
 - Los botones Download y Delete deben habilitarse.
 - Cualquier ítem seleccionado en el sistema de archivos local debe deseleccionarse.
 - El botón Upload debe deshabilitarse.
- Después de ejecutar la operación de subida/descarga necesaria, los botones Upload, Download y Delete deben deshabilitarse. De manera similar, al eliminar el archivo especificado, el botón Delete debe deshabilitarse junto con cualquier botón Upload y Download que esté habilitado en ese momento. Ambos displays de sistemas de archivos (local y remoto) deben actualizarse después de una operación de eliminar, descargar o subir.

A medida que se agregan más controles para funcionalidades adicionales como conectar/desconectar al servidor FTP y otras, la comunicación directa entre objetos crea un complejo laberinto de referencias. El Patrón Mediator puede usarse en este caso para un manejo más eficiente de la interacción entre objetos.

3.3 Clase Mediator

Aplicando el Patrón Mediator, se crea una abstracción para los detalles de interacción. Esta abstracción se diseña como una clase Mediator que ofrece métodos para que los diferentes objetos UI se registren en ella. El conjunto de interacciones a ejecutar cuando se activa cada control UI se diseña como un método separado dentro del Mediator.

Listado 3.1 — Clase Mediator

```
class Mediator {
    private UploadButton btnUpload;
    private DownloadButton btnDownload;
    private DeleteButton btnDelete;
    private LocalList localList;
    private RemoteList remoteList;

    public void registerUploadButton(UploadButton inp_ib) {
        btnUpload = inp_ib;
    }
    public void registerDownloadButton(DownloadButton inp_dnb) {
        btnDownload = inp_dnb;
    }
    public void registerDeleteButton>DeleteButton inp_db) {
        btnDelete = inp_db;
    }
    public void registerLocalList(LocalList inp_arl) {
        localList = inp_arl;
    }
    public void registerRemoteList(RemoteList inp_drl) {
        remoteList = inp_drl;
    }

    public void UploadItem() {
        int index = localList.getSelectedIndex();
        String selectedItem =
            localList.getSelectedValue().toString();
        ((DefaultListModel) localList.getModel()).remove(index);
        ((DefaultListModel) remoteList.getModel()).addElement(selectedItem);
        btnUpload.setEnabled(false);
        btnDelete.setEnabled(false);
        btnDownload.setEnabled(false);
    }

    public void DownloadItem() { /* ... */ }
    public void DeleteItem() { /* ... */ }
    public void LocalListSelect() { /* ... */ }
```

```

    public void RemoteListSelect() {
        localList.setSelectedIndex(-1);
        btnUpload.setEnabled(false);
        btnDelete.setEnabled(true);
        btnDownload.setEnabled(true);
    }
}

```

3.4 Uso del Mediador por el Cliente

El cliente crea una instancia del Mediador. Cuando cada objeto UI es creado, el cliente pasa la instancia del Mediador a ese objeto, que a su vez se registra en la instancia del Mediador.

Listado 3.2 — Clase Cliente FTPGUI

```

public class FTPGUI extends JFrame {
    private Mediator mdtr = new Mediator();

    public FTPGUI() throws Exception {
        // Crear controles
        defLocalList = new DefaultListModel();
        defRemoteList = new DefaultListModel();
        localList = new LocalList(defLocalList, mdtr);
        remoteList = new RemoteList(defRemoteList, mdtr);

        // Crear botones
        UploadButton btnUpload =
            new UploadButton(FTPGUI.UPLOAD, mdtr);
        DownloadButton btnDownload =
            new DownloadButton(FTPGUI.DOWNLOAD, mdtr);
        DeleteButton btnDelete =
            new DeleteButton(FTPGUI.DELETE, mdtr);
    }
} // fin de la clase

```

3.5 Objetos UI Simplificados

Dado que todos los detalles de interacción entre objetos se han movido del interior de los objetos UI al objeto Mediador, el método `processEvent` de cada objeto UI se reduce a una simple llamada al método apropiado del Mediador:

Listado 3.3 — Clases de Objetos UI Simplificadas

```

class UploadButton extends JButton implements CommandInterface {
    Mediator mdtr;

    public UploadButton(String name, Mediator inp_mdtr) {
        super(name);
        mdtr = inp_mdtr;
    }
}

```

```

        mdtr.registerUploadButton(this);
    }
    public void processEvent() {
        mdtr.UploadItem();
    }
}

class DownloadButton extends JButton implements CommandInterface {
    Mediator mdtr;

    public DownloadButton(String name, Mediator inp_mdtr) {
        super(name);
        mdtr = inp_mdtr;
        mdtr.registerDownloadButton(this);
    }
    public void processEvent() {
        mdtr.DownloadItem();
    }
}
// ... (DeleteButton y otras clases siguen el mismo patrón)

```

4. Ejemplo II — Gestión de Biblioteca con Mediador

4.1 El Problema de la Interacción Directa

Durante la discusión del Patrón Comando, se construyó una aplicación para agregar y eliminar ítems en una base de datos de biblioteca. Un ítem dado puede ser parte de una o más categorías. Cada objeto `Item` mantiene una lista de todas las categorías de las que forma parte. De manera similar, cada objeto `Category` mantiene una lista de todos los ítems que forman parte de ella.

Cuando una aplicación debe gestionar muchos ítems pertenecientes a muchas categorías, las interacciones entre objetos pueden volverse complicadas, con diferentes objetos `Item` y `Category` referenciándose directamente entre sí.

4.2 Diseño del Mediador

La interacción directa entre los diferentes objetos `Item` y `Category` puede eliminarse moviendo los detalles de interacción fuera de las clases `Item` y `Category` hacia una clase `Mediator` separada. El `Mediator` puede diseñarse con los siguientes dos conjuntos de métodos:

- Un conjunto de métodos para permitir que los diferentes objetos `Item` y `Category` se registren en el `Mediator`.
- Un conjunto de métodos para agregar y eliminar ítems. El `Mediator` es responsable de implementar las interacciones entre los diferentes objetos a través de estos métodos.

El `Mediator` puede mantener la asociación Ítem-Categoría en una variable de instancia `CatAssoc` (de tipo `HashMap`). Los objetos `Item` no necesitan hacer referencia directa a objetos

Category. Por lo tanto, un objeto Item no necesita mantener las categorías a las que pertenece, y viceversa. De manera similar, las operaciones de agregar y eliminar no necesitan ser implementadas por las clases Item y Category.

El método execute de los objetos Comando AddCommand y DeleteCommand se reduce a una llamada a los métodos addItem y deleteItem del Mediador, respectivamente.

Listado 4.1 — Clase Mediador para Gestión de Biblioteca

```
class Mediator {
    private HashMap itemCatAssoc;

    public void registerItem(Item i) {
        // registrar el item en el mediador
    }

    public void registerCategory(Category c) {
        // registrar la categoria en el mediador
    }

    public void add(Category c, Item i) {
        // El Mediador coordina la interaccion
        // entre Item y Category para agregar
        itemCatAssoc.put(i.getDesc(), c);
        c.add(i);
        i.add(c);
    }

    public void delete(Category c, Item i) {
        // El Mediador coordina la interaccion
        // entre Item y Category para eliminar
        itemCatAssoc.remove(i.getDesc());
        c.delete(i);
        i.delete(c);
    }
}
```

5. Comparación: Comunicación Directa versus con Mediador

Tabla 3 — Comunicación Directa versus Comunicación con Mediador

Aspecto	Sin Mediador	Con Mediador
Acoplamiento entre objetos	Alto: cada objeto referencia directamente a los demás con quienes interactúa.	Bajo: los objetos solo conocen al Mediador, no entre sí.

Aspecto	Sin Mediador	Con Mediador
Mantenibilidad	Baja: modificar las interacciones requiere cambios en múltiples clases.	Alta: las interacciones se modifican únicamente en la clase Mediador.
Reutilizabilidad	Reducida: las dependencias directas dificultan reutilizar objetos individualmente.	Mejorada: los objetos pueden reutilizarse en otros contextos al no depender directamente unos de otros.
Pruebas unitarias	Difíciles: un objeto no puede probarse sin instanciar todos los objetos con los que interactúa.	Más sencillas: cada objeto puede probarse de forma independiente.
Complejidad del Mediador	No aplica.	El Mediador puede volverse complejo si concentra demasiadas interacciones; puede mitigarse con subclasificación.

6. Preguntas de Práctica

1. Los representantes de servicio al cliente de algunos bancos comerciales reciben consultas de sus clientes actuales y potenciales a través de una aplicación de chat en línea. En horas pico, cada representante puede necesitar atender a más de un cliente simultáneamente. Diseñe este mecanismo de comunicación con un objeto Mediador entre los diferentes objetos Usuario y los objetos Representante.
2. Implemente la aplicación del Ejemplo II.